

GitHub Copilot Tutorial for Students

Practical workflows for VS Code, prompt design, model selection, and responsible AI-assisted programming

Prepared for classroom use

Last updated: June 26, 2026

How to Use This Tutorial

This tutorial is written for students who are learning to use GitHub Copilot as a programming assistant inside Visual Studio Code. The goal is not to make Copilot do your homework for you. The goal is to learn how to ask better questions, read generated code critically, plan before coding, and use AI support without giving up responsibility for the final work.

You should read this document with VS Code open. Try the shortcuts, ask the prompts, and compare Copilot's answers against your course notes, compiler errors, tests, and documentation. Copilot can be very helpful, but it can also be confidently wrong. Treat it as a fast pair programmer, not as an authority.

This tutorial focuses on five habits:

- Start with a plan before asking for code.
- Give Copilot enough context to understand the task.
- Choose the right mode for the job: Ask, Edit, Agent, or planning.
- Choose the right model size for the job.
- Verify every important answer with tests, reasoning, and review.

The examples use Windows and Linux keyboard shortcuts. On macOS, Ctrl is often replaced by Command, and some shortcuts differ. If a shortcut does not work, open the Command Palette with Ctrl+Shift+P and search for the command by name.

By the end, you should be able to:

- Open and use Copilot Chat in VS Code.
- Use Ctrl+Shift+P to find commands quickly.
- Use Ctrl+Shift+I and inline editing to work directly inside a file.
- Explain Ask mode, Edit mode, Agent mode, and plan-first workflows.
- Select models based on task difficulty and cost.
- Write prompts with context, instructions, and guardrails.
- Add repository instructions in `.github/copilot-instructions.md`.
- Understand how MCP servers and agent skills can provide extra context and tools.

The most important idea is simple: planning is part of programming. Do not save planning for large projects only. Even a short programming problem benefits from a clear plan, assumptions, inputs, outputs, and tests.

1. What GitHub Copilot Is

GitHub Copilot is an AI programming assistant. In VS Code, it can suggest code as you type, answer questions about files, explain errors, help write tests, propose refactors, and carry out multi-step tasks when used in agentic modes. It works best when you give it a clear task and enough project context.

Copilot is not one feature. It is a collection of related features:

- Code completions: suggestions that appear while you type.
- Chat: a conversational panel for questions and tasks.
- Inline chat and inline editing: prompts entered directly in the editor.
- Edit mode: targeted code changes to selected files or regions.
- Agent mode: multi-step work where Copilot can inspect files, propose edits, and sometimes run commands or tools with approval.
- Custom instructions: persistent rules that tell Copilot how to work in a project.
- MCP tools: external context and actions exposed through Model Context Protocol servers.

For students, the danger is using Copilot as a replacement for thinking. If you ask, "Write my assignment," you may get code that seems plausible but does not match the lesson, the style requirements, the hidden tests, or your own understanding. A better request is: "Help me design a plan for this assignment. Ask me clarifying questions first. Do not write the final code yet."

Copilot is strongest when you already have a direction. It can reduce friction, explain unfamiliar syntax, generate examples, and help you get unstuck. It is weakest when the task is vague, the context is missing, or you accept the first answer without review.

A productive mental model is:

- You are the driver.
- Copilot is the navigator and assistant.
- The compiler, tests, and course requirements are the judge.

When these disagree, do not trust the most fluent answer. Investigate.

Classroom Rule

Any submitted code must be code you can explain. If Copilot produced a function, you should be able to describe its inputs, outputs, control flow, edge cases, and tests. If you cannot explain it, you have not finished learning from it.

2. Setup in VS Code

Before using Copilot, make sure your environment is ready.

1. Install Visual Studio Code.
 2. Sign in to GitHub from VS Code.
 3. Install or enable the GitHub Copilot and GitHub Copilot Chat extensions if your VS Code installation does not include them.
 4. Open a folder, not just a single file, so Copilot can see project context.
 5. Confirm your course repository is open and trusted.
- In a student workflow, you usually want one VS Code window per assignment repository. This gives Copilot a clearer workspace and keeps context from different projects from mixing together.

Recommended layout:

- Explorer on the left.
- Editor in the center.
- Terminal at the bottom.
- Copilot Chat in the secondary sidebar or panel.
- Source Control view available for reviewing changes.

Good setup also includes a working test command. For example:

```
npm test
pytest
mvn test
dotnet test
cargo test
```

If you do not know how to run the project, ask Copilot in Ask mode:

Look at this repository and tell me how to run the tests. Do not change files.
Then verify the answer in the README, package files, build files, or course instructions. Copilot may infer a command that is common but wrong for your specific repository.

First Safety Habit

Before asking Copilot to edit code, check Git status:

```
git status
```

Know what changed before and after AI assistance. In VS Code, use the Source Control view to inspect every modification. Never commit generated code that you have not reviewed.

3. The Command Palette: Ctrl+Shift+P

The most important VS Code shortcut is Ctrl+Shift+P. It opens the Command Palette. The Command Palette is a searchable list of nearly everything VS Code can do. If you forget a shortcut, use the Command Palette.

Examples of useful searches:

- GitHub Copilot: Open Chat
- Chat: Focus on Chat View
- Copilot: Explain
- Copilot: Fix
- Copilot: Generate Tests
- Preferences: Open Keyboard Shortcuts
- Developer: Reload Window
- Git: Clone
- Git: Commit
- Terminal: Create New Terminal

Use Ctrl+Shift+P when:

- A keyboard shortcut is different on your machine.
- A Copilot button is hidden.
- You want to discover related commands.
- You need to reset, reload, or configure VS Code.

The Command Palette matters because it teaches you the names of actions. Once you know the command name, you can find documentation, assign a custom shortcut, or ask Copilot about it precisely.

Weak prompt:

Copilot is not working.

Better prompt:

In VS Code, I opened the Command Palette with Ctrl+Shift+P and ran GitHub Copilot: Open Chat, but the chat panel did not appear. What should I check next? Ask me for version or sign-in details if needed.

The better prompt gives Copilot the exact action, expected result, actual result, and permission to ask clarifying questions.

Practice

Open the Command Palette and search for Copilot. Do not run every command. Read the names. Notice which actions are about chat, which are about completions, which are about fixes, and which are about settings.

4. Copilot Chat and Ctrl+Shift+I

Copilot Chat is the main place to ask questions, request explanations, and work through tasks. In VS Code, you can use chat in several ways:

- Chat view: a panel or sidebar conversation.
- Quick chat: a small prompt box for fast questions.
- Inline chat: a prompt directly in the editor.
- Smart actions: context menu actions such as explain, fix, or generate tests.

Ctrl+Shift+I is commonly used to start inline Copilot Chat in the editor. Inline chat is useful when you want Copilot to focus on the current file or selected code. If that shortcut does not work in your setup, use Ctrl+Shift+P and search for inline chat or Copilot inline commands.

Use inline chat when:

- You selected a block of code and want it explained.
- You want a small edit in one place.
- You want to ask about an error near the cursor.
- You want Copilot to rewrite a function without leaving the editor.

Example inline prompt:

```
Explain what this function does in plain English. Then identify one possible edge case. Do not  
edit the file.
```

Example inline edit prompt:

```
Refactor this function to use early returns. Keep the same behavior. Do not rename public  
functions. Add no new dependencies.
```

Inline chat is not the best place for broad architecture decisions. Use the full Chat view for larger conversations because it is easier to keep track of context, plans, and follow-up questions.

Selection Matters

When using inline chat, select the smallest relevant code region. If you select too little, Copilot may miss important dependencies. If you select too much, it may make broad edits. For a bug in one function, select the function. For a bug involving two functions, select both or reference the file names in chat.

5. Code Completions

Code completions are suggestions that appear as you type. They are different from chat. A completion is usually based on the current file, nearby code, comments, names, and project context.

Completions are helpful for:

- Repetitive code.
- Boilerplate.
- Common syntax.
- Test cases that follow an existing pattern.
- Small helper functions.

Completions are risky for:

- Security-sensitive code.
- Authentication and authorization.
- Database queries.
- Concurrency.
- Numerical algorithms.
- Code you do not understand.

To get better completions, write clear names and comments before the code. Copilot responds strongly to surrounding context.

Poor setup:

```
def f(x):  
    pass
```

Better setup:

```
def normalize_email(raw_email: str) -> str:  
    """Return a lowercase, stripped email address. Raise ValueError if missing @."""
```

Now Copilot has a name, type information, behavior, and error rule. The resulting suggestion is more likely to be useful.

Do not accept large completions blindly. Accept a few lines, read them, then continue. If a suggestion is wrong, reject it and write more context yourself. You are allowed to guide the assistant by writing better code around it.

Practice

Create a small function with a descriptive name and docstring. Pause and watch the completion. Then reject it and rewrite the docstring with clearer constraints. Notice how the suggestion changes.

6. Ask Mode

Ask mode is for conversation and explanation. It should be your default when you are uncertain. In Ask mode, Copilot answers without directly changing your files unless you explicitly move into an editing flow.

Use Ask mode for:

- Understanding an error message.
- Explaining unfamiliar code.
- Comparing two approaches.
- Asking what command runs tests.
- Learning a library concept.
- Requesting a plan before implementation.
- Reviewing a solution you wrote yourself.

Good Ask mode prompt:

```
I am learning recursion. Explain this function step by step using the input [1, 2, 3]. Do not  
rewrite the code yet. After explaining, ask me one question to check my understanding.
```

Ask mode is excellent for small models because many questions do not require the most expensive or most powerful model. A small or fast model is usually enough for definitions, syntax explanations, short examples, and quick comparisons. Save larger models for tasks where reasoning quality matters more.

Ask mode is also the safest mode for academic integrity. It lets you learn without immediately generating a final answer. When in doubt, ask Copilot to teach, question, or review rather than solve.

Ask Mode Guardrails

Useful guardrails:

- "Do not write the final code."
- "Ask me clarifying questions first."
- "Explain using beginner-friendly language."
- "Point to the exact line that causes the error."
- "Give hints in three levels, from subtle to direct."
- "Use the concepts from this week's lesson only."

These guardrails keep the interaction educational.

7. Edit Mode and Inline Editing

Edit mode is for targeted changes. Instead of merely answering, Copilot proposes edits to files. Inline editing is the smaller version of this idea: you select code, ask for a change, and review the diff.

Use Edit mode for:

- Refactoring a function.
- Adding input validation.
- Writing tests for an existing module.
- Updating names for clarity.
- Fixing a localized bug.
- Converting a loop to a clearer structure.

Do not use Edit mode when you have not decided what should change. If the problem is still vague, use Ask mode or planning first.

Good edit prompt:

```
In 'src/cart.py', update 'calculate_total' so it ignores inactive items and applies the discount
after tax. Preserve the public function signature. Add or update unit tests. Do not change
unrelated files.
```

Notice the ingredients:

- File name.
- Function name.
- Desired behavior.
- Constraint on public API.
- Test instruction.
- Scope guardrail.

After Copilot proposes edits, review the diff. Ask:

- Did it change only the intended files?
- Did it preserve names and interfaces required by tests?
- Did it add unnecessary dependencies?
- Did it handle edge cases?
- Did it update or add tests?
- Can I explain every changed line?

If the answer is no, do not accept the edit as finished. Ask for a smaller change or manually correct it.

Inline Editing Prompt Pattern

Use:

```
Change [specific code] so that [desired behavior]. Keep [constraints]. Do not [forbidden
changes]. Add [tests or comments only if needed].
```

This pattern prevents broad, surprising rewrites.

8. Agent Mode

Agent mode is for multi-step tasks. In agentic workflows, Copilot can inspect files, decide an approach, propose edits, and sometimes use tools such as terminal commands, tests, or MCP tools. Depending on your setup and policies, it may ask for confirmation before running commands or applying changes.

Use Agent mode when:

- The task spans multiple files.
- The implementation requires exploration.
- You want Copilot to run or interpret tests.
- You need a coordinated refactor.
- You want a bug investigated from symptoms.
- You need generated code plus validation.

Do not use Agent mode for every small question. It is more powerful, but also easier to over-scope.

If you only need an explanation, use Ask mode. If you need one function changed, use Edit mode. If you need a multi-file investigation, use Agent mode.

Good Agent mode prompt:

```
Investigate why the checkout tests are failing. First inspect the relevant tests and
  implementation. Then propose a plan before editing. Do not change public APIs unless the
  tests require it. After edits, run the smallest relevant test command and summarize the
  result.
```

Agent mode should still be supervised. Read its plan. Watch which files it opens. Check commands before approving them. Review the final diff.

Agent Mode Risks

Agent mode can make confident but broad changes. It may fix the symptom while weakening design, remove tests instead of fixing code, or introduce dependencies that are not allowed. Guardrails matter more as autonomy increases.

Good guardrails:

- "Do not delete tests."
- "Do not add dependencies."
- "Use the existing project style."
- "Stop and ask before changing database schema."
- "Make the smallest change that fixes the failing test."
- "Show a plan and wait before editing."

9. Plan Mode and Plan-First Work

Plan mode means using Copilot to reason about the approach before writing code. Some Copilot surfaces provide a plan-oriented workflow directly. Even when there is no separate button labeled Plan, you can create a plan-first workflow by prompting for a plan and telling Copilot not to edit yet.

Planning is the most important part of writing code because it determines what problem you are solving. A weak plan leads to wasted implementation. A strong plan makes coding faster, testing clearer, and review easier.

Use planning for:

- Assignments with ambiguous requirements.
- Multi-file features.
- Debugging unfamiliar code.
- Refactors.
- Database changes.
- API design.
- Security-sensitive work.
- Any task where a wrong direction would be expensive.

Planning prompt:

Before writing code, create a plan for implementing this assignment. Include assumptions, files likely to change, edge cases, and tests. Ask clarifying questions if requirements are missing. Do not edit files yet.

A good plan should include:

- Goal.
- Current understanding.
- Assumptions.
- Proposed file changes.
- Data flow.
- Edge cases.
- Test plan.
- Risks.
- Checkpoints.

Planning is not wasted time. It is how you reduce rework.

Classroom Planning Standard

Before using Agent mode for an assignment, write or request a plan. Then revise it. If the plan does not mention tests, edge cases, or constraints, it is not done.

For major tasks, use a large model for planning. The extra reasoning quality matters because mistakes in the plan cascade into every later step.

10. Which Model Should You Select?

Copilot offers multiple models, and the available list changes over time by plan, IDE, organization policy, and product rollout. Do not memorize a single permanent list. Instead, learn a selection strategy.

This course recommends a size-by-task rule:

- Small models for asking.
- Medium models for writing code.
- Large models for planning and hard reasoning.

Why?

Small models are usually fast and economical. They are good for direct questions, syntax help, simple explanations, quick examples, and summarizing a short file.

Medium models are a good default for normal implementation. They can write code, follow project style, and handle moderate context without using the highest-cost option.

Large models should be saved for planning, architecture, debugging hard failures, security review, and multi-step reasoning. Planning deserves the strongest model because it shapes the whole solution. A weak implementation can be corrected locally; a weak plan can send the entire project in the wrong direction.

Practical Mapping

Use a small model when asking:

What does this Python error mean?
Explain the difference between a list and a tuple.
Show a tiny example of a SQL join.

Use a medium model when coding:

Implement this helper function using the existing style.
Write unit tests for this module.
Refactor this function without changing behavior.

Use a large model when planning:

Design a plan to add authentication to this app.
Analyze this failing integration test and propose likely causes.
Review this architecture for security and maintainability risks.

If your organization offers auto model selection, it may choose a model for you. Even then, understand the tradeoff. Bigger is not always better. The right model is the smallest one that can do the job well.

11. Model Selection Checklist

Before choosing a model, ask four questions.

First, how much reasoning is required? If the task involves tradeoffs, hidden dependencies, or multiple files, use a stronger model. If it is a short factual or syntax question, use a smaller one.

Second, how much context is required? If Copilot must understand a large repository, a model with a larger context window may help. But more context can also distract the model if you include irrelevant files.

Third, what is the cost or quota impact? Some models consume more credits or premium requests. Students should learn to budget model use. Do not spend a large model on questions a small model can answer.

Fourth, what is the risk of being wrong? A wrong explanation in a practice exercise is easy to correct. A wrong security fix, migration plan, or database change is expensive. Higher risk justifies stronger reasoning and more human review.

Recommended Defaults

For this class:

- Ask mode default: small or fast model.
- Edit mode default: medium coding model.
- Agent mode default: medium model for routine tasks, large model for hard tasks.
- Plan-first prompt: large model.
- Security review: large model, plus human review and tests.
- Final polish or formatting: small or medium model.

Do not rely only on model names. Providers and model families change. Look at the model picker, course policy, and GitHub's supported model documentation. If a model is retired or unavailable, choose the closest available size and capability.

Important Habit

When switching models, tell Copilot why:

Use a planning-oriented answer. I am selecting a larger model because this is a multi-file design task. Do not write code yet.

This gives the model a role and reduces the chance it jumps into implementation too early.

12. Prompt Design: Context, Instructions, Guardrails

A strong prompt has three parts:

- Context: what Copilot needs to know.
- Instructions: what you want done.
- Guardrails: what must not happen or what constraints must be followed.

Weak prompt:

Fix this.

Strong prompt:

Context: This is a Flask route that handles new user registration. The failing test says duplicate emails should return HTTP 409.

Instructions: Find the cause and update the route so duplicate emails are rejected.

Guardrails: Do not change the database schema. Do not remove tests. Keep the JSON response format used by the existing error handlers. Add a focused unit test if one is missing.

Context tells Copilot where it is. Instructions tell it what to do. Guardrails prevent accidental damage.

Good context can include:

- The assignment requirement.
- File names.
- Error messages.
- Test output.
- Relevant code.
- Data examples.
- Course constraints.
- What you already tried.

Good instructions include:

- "Explain."
- "Plan."
- "Refactor."
- "Implement."
- "Write tests."
- "Compare options."
- "Review for bugs."

Good guardrails include:

- "Do not edit files yet."
- "Do not add dependencies."
- "Use beginner-level Python."
- "Keep the public API unchanged."
- "Use only topics covered in class."
- "Ask questions if requirements are unclear."

Prompt design is not about magic wording. It is about making the task unambiguous.

13. Prompt Templates for Students

Use these templates as starting points. Replace bracketed text with your details.

Understanding Code

I am learning [topic]. Explain [file/function] step by step. Use beginner-friendly language. Do not rewrite the code. After explaining, identify two edge cases and ask me one comprehension question.

Debugging

Context: I expected [expected behavior], but I got [actual behavior]. The error message is [paste error]. The relevant files are [files].

Instructions: Help me diagnose the likely cause. Start with a short list of hypotheses, then tell me what to inspect first.

Guardrails: Do not edit code yet. Do not guess if the evidence is missing; ask me for the missing output.

Planning

Create a plan for [task]. Include assumptions, files likely to change, implementation steps, edge cases, and tests. Do not write code yet. If the requirements are ambiguous, ask clarifying questions first.

Editing

Change [file/function] so that [desired behavior]. Preserve [interfaces/style]. Do not [forbidden changes]. Add or update tests for [cases]. Keep the diff small.

Reviewing

Review this change for correctness, readability, edge cases, and tests. Prioritize bugs over style. Do not rewrite the code unless you find a concrete issue.

Learning Without Cheating

Give me hints for solving this problem. Start with a conceptual hint, then a more specific hint, then pseudocode only if I ask. Do not provide the final code.

Templates reduce cognitive load. Over time, you should adapt them to the course, language, and assignment style.

14. Giving Copilot the Right Context

Copilot can use open files, selected code, repository context, chat history, custom instructions, and sometimes tool-provided context. But it does not automatically know everything you know.

Good context is relevant, current, and specific.

Relevant means you include the files, errors, and requirements that matter. Do not paste the whole project when one function and one test are enough.

Current means you provide the latest error output after your most recent change. Old error messages can mislead the assistant.

Specific means you name files, functions, inputs, and outputs.

Weak context:

My tests fail.

Better context:

```
'tests/test_cart.py::test_discount_after_tax' fails. It expected 10.80 but got 10.50. The
  relevant function is 'calculate_total' in 'src/cart.py'. The assignment says tax is applied
  before discount. Explain the likely issue before suggesting edits.
```

Context Tools in VS Code

Depending on your VS Code and Copilot version, you may be able to add files, folders, symbols, terminal output, or other context to a prompt. Use these features deliberately. More context is not always better. A focused prompt with one file and one failing test often beats a vague prompt with the whole repository.

Keep History Relevant

Long chat history can confuse the model. Start a new chat when:

- You changed tasks.
 - The model is stuck in a wrong assumption.
 - The conversation includes outdated errors.
 - You switched from learning to implementation.
- In the new chat, summarize only the useful facts.

15. Repository Instructions: `.github/copilot-instructions.md`

Repository instructions are persistent guidance stored in the repository. In VS Code and GitHub, a common file is:

`.github/copilot-instructions.md`

This file tells Copilot how to behave when working in the project. It is useful because students otherwise repeat the same constraints in every prompt.

Example:

```
# Copilot Instructions
```

- Use Python 3.12 syntax unless the assignment says otherwise.
- Prefer clear beginner-readable code over clever one-liners.
- Do not add third-party dependencies without asking.
- Preserve public function signatures used by tests.
- When changing behavior, add or update unit tests.
- Do not remove failing tests to make the suite pass.
- Explain assumptions before making broad edits.

Good repository instructions are short, specific, and stable. Do not put the entire assignment in this file if it changes every week. Put durable project rules there.

Path-specific instructions may also be supported through files such as:

`.github/instructions/python.instructions.md`

`.github/instructions/tests.instructions.md`

These can guide Copilot differently for application code and tests. For example, test instructions might say to use pytest, avoid sleeps, and prefer deterministic fixtures.

What Not to Put in Instructions

Do not put secrets, passwords, API keys, private student information, or temporary debugging notes in repository instructions. They are project files and may be committed.

Also avoid contradictory instructions. If one instruction says "always write detailed comments" and another says "avoid comments," Copilot may behave inconsistently.

16. MCP Servers and Agent Skills

MCP stands for Model Context Protocol. It is an open standard for connecting AI tools to external context and capabilities. In Copilot, MCP servers can provide tools or resources that Copilot can use, especially in Agent mode.

Examples of what MCP servers can expose:

- Repository or issue data.
- Browser or web-fetching tools.
- Database inspection tools.
- Documentation search.
- Internal APIs.
- File or project resources.
- Testing or automation tools.

Think of MCP as a structured way to give Copilot safe access to tools beyond the text you type. Instead of pasting a database schema into chat, a configured MCP server might provide schema context. Instead of manually copying issue details, a GitHub MCP server might expose issue and pull request data.

Agent skills are reusable instructions or capabilities that guide agent behavior for a domain. A skill might teach an agent how to run a project, follow a deployment process, review accessibility, or use a team-specific checklist.

For students, the key idea is:

- Custom instructions tell Copilot how to behave.
- MCP servers give Copilot additional context or tools.
- Skills package specialized workflows.

Safety With MCP

Tools increase power and risk. Before allowing an MCP tool or agent action, ask:

- What data can this tool read?
- What can it write or change?
- Does it require secrets?
- Is it approved for this course or organization?
- Can I inspect what it did afterward?

Use only MCP servers your instructor or organization approves. A tool-connected assistant should be treated like a developer with access, not like a harmless autocomplete box.

17. A Student Workflow for Assignments

Here is a disciplined workflow for using Copilot on programming assignments.

Step 1: Read the Assignment Yourself

Before opening Copilot, identify:

- Required inputs and outputs.
- Files you are expected to edit.
- Concepts the assignment is practicing.
- Restrictions on libraries or style.
- How the work will be tested.

Step 2: Ask for Clarification, Not Code

Prompt:

I will paste an assignment description. Summarize the requirements, list ambiguities, and ask clarifying questions. Do not write code.

Step 3: Request a Plan

Use a large model if available:

Create a plan for this assignment. Include data structures, functions, edge cases, and tests. Keep the plan appropriate for an introductory programming course.

Step 4: Implement in Small Pieces

Use completions or Edit mode for one function at a time. After each change, run tests.

Step 5: Ask for Review

Prompt:

Review my solution for bugs and missing edge cases. Do not rewrite it. Point to specific lines and explain the issue.

Step 6: Explain It Back

Ask Copilot to quiz you:

Ask me five questions about this solution that would prove I understand it. If you cannot answer, keep studying before submitting.

18. Debugging With Copilot

Debugging is one of the best uses of Copilot, but only if you provide evidence.

A good debugging prompt includes:

- Expected behavior.
- Actual behavior.
- Exact error message.
- Relevant input.
- Relevant files or function names.
- What you already tried.
- Whether you want explanation, plan, or edits.

Example:

Expected: `'parse_date("2026-06-26")'` returns a date object.

Actual: it raises `'ValueError: time data does not match format'`.

Relevant file: `'src/dates.py'`.

I already checked that the input is a string.

Explain the likely cause and suggest one minimal fix. Do not edit yet.

Ask Copilot to form hypotheses before editing. This keeps debugging from turning into random changes.

List three possible causes ranked by likelihood. For each, tell me what evidence would confirm or disprove it.

Then inspect the evidence. This is how professional debugging works: hypothesis, evidence, experiment, conclusion.

Do Not Let Copilot Hide the Bug

A bad AI fix may catch every exception and return a default value. That can make tests pass while hiding real errors. Guard against this:

Do not silence exceptions unless the assignment explicitly requires it. Prefer fixing the root cause.

After a fix, run the smallest relevant test first, then the full suite. If no tests exist, ask Copilot to help design tests before trusting the fix.

19. Testing With Copilot

Copilot can help write tests, but you must decide what behavior matters.

Good test prompt:

```
Write pytest tests for 'calculate_total'. Cover empty cart, inactive items, discount after tax,
and invalid negative prices. Use the existing test style. Do not change production code.
```

Weak test prompt:

```
Write tests.
```

Strong tests include normal cases, edge cases, and failure cases. Copilot often writes happy-path tests first. Push it further.

Ask:

```
What edge cases are missing from these tests?
```

or:

```
Review these tests. Are they testing behavior or only implementation details?
```

Students should learn the difference between:

- Testing that code runs.
- Testing that code is correct.
- Testing that code handles edge cases.
- Testing that code fails safely.

Test-First Prompt

For some assignments, start with tests:

```
Based on this requirement, propose test cases before implementation. Do not write production
code. Include expected inputs and outputs.
```

This is another form of planning. If you cannot describe tests, you probably do not understand the requirement yet.

Never Remove Tests Casually

If Copilot suggests deleting or weakening a failing test, stop. Ask:

```
Explain why this test is wrong according to the requirements. If it is valid, fix the
implementation instead.
```

20. Code Review and Academic Integrity

Copilot can review code, but it does not replace your own review. Use it as a second reviewer.

Good review prompt:

Review this diff for correctness, edge cases, readability, and tests. Prioritize real bugs. Do not comment on style unless it affects maintainability.

Ask for severity:

Classify findings as high, medium, or low severity. Include file and line references when possible.

For academic integrity, follow your course policy. If AI assistance is allowed, you should still be transparent when required. Keep a record of major prompts if your instructor asks for them.

You are responsible for:

- Understanding submitted code.
- Following assignment constraints.
- Citing or reporting AI assistance if required.
- Avoiding unauthorized collaboration.
- Not submitting generated work you cannot explain.

Better Learning Prompts

Instead of:

Solve this assignment.

Use:

Help me understand the problem and design a plan.

Instead of:

Write the code for me.

Use:

Give me pseudocode and ask me to implement the next step.

Instead of:

Make it pass.

Use:

Explain why the test fails and guide me to the minimal fix.

Copilot should accelerate learning, not bypass it.

21. Common Mistakes

Mistake 1: Asking Vague Questions

"Fix my code" gives Copilot too much freedom. Name the file, function, expected behavior, and constraints.

Mistake 2: Skipping the Plan

Students often jump into code because code feels productive. Planning prevents wasted effort. Use large models for planning when the task matters.

Mistake 3: Accepting Large Diffs

A huge diff is hard to review. Ask for smaller changes:

Make only the smallest change needed for this test. Do not refactor unrelated code.

Mistake 4: Trusting Explanations Without Running Code

Copilot can explain code that does not compile. Always run tests or execute examples.

Mistake 5: Letting Copilot Change the Assignment

If the assignment says no third-party libraries, do not accept a solution that adds one. If it requires recursion, do not accept a loop-only solution unless the instructor permits it.

Mistake 6: Hiding Lack of Understanding

If you cannot explain the code, ask Copilot to teach:

Explain this solution line by line and then quiz me.

Mistake 7: Using Agent Mode Without Guardrails

Agent mode needs boundaries. Tell it what files not to touch, what tests to run, and when to stop.

Mistake 8: Confusing More Context With Better Context

More files can help, but irrelevant files can distract. Provide the smallest sufficient context.

22. End-to-End Example

Scenario: A function should count words in a sentence, ignoring punctuation and case. The current implementation fails on "Hello, hello!".

Ask Mode

```
I am writing a beginner Python function that counts words case-insensitively and ignores
punctuation. The input "Hello, hello!" should count hello twice. Explain what edge cases I
should consider. Do not write code.
```

Expected result: Copilot lists cases such as punctuation, capitalization, empty strings, repeated spaces, and numbers.

Plan Mode

Create a plan before coding. Include steps, edge cases, and tests. Use only Python standard

library. Do not write final code yet.

Expected result: A plan involving normalization, punctuation handling, splitting, counting, and tests.

Edit Mode

Update 'count_words' in 'word_count.py' according to the plan. Use only the standard library. Preserve the function signature. Add pytest tests for punctuation, capitalization, empty input, and repeated spaces.

Expected result: A small diff touching the function and tests.

Review

Review the diff. Check whether the tests prove the required behavior. Identify any edge cases still missing. Do not rewrite unless there is a bug.

Student Explanation

Finally, explain the code yourself:

- How is punctuation removed?
- How is case normalized?
- What happens with an empty string?
- What does the function return?
- Which test would fail if case normalization were removed?

If you can answer these, Copilot helped you learn. If not, keep working.

23. Quick Reference

Shortcuts

- Ctrl+Shift+P: Open the VS Code Command Palette.
- Ctrl+Shift+I: Open inline Copilot Chat in many VS Code setups.
- Ctrl+P: Quickly open a file by name.
- Ctrl+': Toggle the integrated terminal.
- Ctrl+B: Toggle the sidebar.

Mode Selection

- Ask mode: questions, explanations, debugging help, review without edits.
- Edit mode: targeted changes to selected code or specified files.
- Agent mode: multi-file, multi-step work with supervised autonomy.
- Plan-first workflow: use before implementation, especially for complex tasks.

Model Selection

- Small model: quick questions and explanations.
- Medium model: normal coding and tests.
- Large model: planning, architecture, hard debugging, security review.

Prompt Formula

Context: [what Copilot needs to know]

Instructions: [what you want done]

Guardrails: [constraints and forbidden changes]

Before Accepting AI Code

- Run tests.
- Read the diff.
- Check assignment constraints.
- Confirm no unrelated files changed.
- Explain the code in your own words.
- Commit only after review.

Best Student Prompt

Help me learn this. Guide me with questions and hints before giving code.

References and Further Reading

These official documentation pages are useful starting points. Copilot changes frequently, so always check current documentation when model lists, policies, shortcuts, or feature names matter.

- GitHub Docs: Chat in IDE -
<https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
- GitHub Docs: Supported AI models in GitHub Copilot -
<https://docs.github.com/en/copilot/reference/ai-models/supported-models>
- GitHub Docs: Prompt engineering for GitHub Copilot Chat -
<https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering>
- GitHub Docs: About customizing GitHub Copilot responses -
<https://docs.github.com/en/copilot/concepts/prompting/response-customization>
- GitHub Docs: About Model Context Protocol (MCP) -
<https://docs.github.com/en/copilot/concepts/context/mcp>
- GitHub Docs: Extending GitHub Copilot Chat with MCP servers -
<https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/extend-copilot-chat-with-mcp>
- VS Code Docs: User interface and Command Palette -
<https://code.visualstudio.com/docs/editing/userinterface>

Final Advice

Use Copilot to become more deliberate, not less deliberate. Ask for plans. Ask for explanations. Ask for reviews. Use small models for simple questions, medium models for normal coding, and large models for planning because planning determines the quality of the work that follows.

The best Copilot users are not people who type the fewest characters. They are people who give clear context, set useful constraints, review carefully, and keep responsibility for the final result.